SMARTWIZ+ art API Developer Guide

1. Overview

This document explains how to use the APIs provided by the SMARTWIZ+ art device.
The SMARTWIZ+ art device communicates with external devices via BLE and HTTP.
Hereafter, the SMARTWIZ+ art device will be referred to as the “art frame.”

2. BLE Communication

For BLE communication, art frame uses the BLUFI protocol.
For more information about BLUFI, please refer to the following link:
https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/ble/blufi.html

WiFi connection/disconnection, SSID scanning, and retrieval of scan results follow the Blufi specification.
For details on using Blufi, please refer to Section 4: Usage and the sample scripts.

2.1. Detection and connection of the Art Frame

The Art Frame advertises using the local name:
SMARTWizArt-<PID>.
Use this local name to connect via Blufi.
The PID is a 10-digit number displayed on the screen when the device is initialized.

2.2. Custom Data API

Art Frame implements an API using Blufi's Custom Data Send/Receive functionality.
Data is sent and received in JSON format.

2.2.1. Common request_fields

The following fields are common to all calls of the Custom Data API.

Field	Data Type	Required	Description
command	string	Yes	User's public key encoded in Base64.
args	object	Yes	API parameters defined for each command to be called. If there are no parameters, specify null.
agent	string	Yes	Name of the user application calling the API.

2.2.1. Custom Data API List

No	Command	Description
1	get_device_info_request	Retrieves information about the art frame.

2.2.2. get_device_info_request

(1). **Args**

None

(2). **Response**

Field	Data Type	Description
result	boolean	Result of the API operation: true: success false: failure
device_id	object	Art Frame device ID
version	string	Art Frame firmware version
model	string	Product model Name
hostname	string	Art Frame hostname
api_url	string	HTTP API base url
wifi_connected	Boolean	WiFi Connect status true: Connected false: Not Connected
wifi_ssid	string	Connected Wifi SSID
is_matter_available	boolean	Matter commissioning status true: Commissioned false: Not commissioned
ipv4_addr	string	Art Frame IPV4 address
ipv6_addr	string	Art Frame IPV4 address
owner_hash	string	Character string used to check the user application that has been registered with the device.

3. HTTP Communication

HTTP communication allows device registration and image uploads for the Art Frame.

3.1. Device Registration and Security

The images for the art frame must be encrypted with art frame's public key in order to protect privacy. In addition, HTTP request signatures must be generated using the private key on the client side that calls the API, and the art frame verifies the signature using the corresponding public key.

Therefore, the user needs to prepare a private key and a public key.

The specifications and example commands for generating them are as follows.

Encryption Algorithm	RSA
key length	2048
Certificate Format	DER

```
# generate RSA 2048 key-pair
openssl genpkey -algorithm RSA -out app_private.der -pkeyopt rsa_keygen_bits:2048 -outform DER
openssl rsa -pubout -in app_private.der -inform DER -out app_public.der -outform DER
```

The OpenSSL command is used to generate the certificate.Please install the required package if necessary.

3.1.1. Device Registration

To use the art frame's features via the API, it is necessary to exchange the art frame's public key and the user's public key using "device_register_request".

For details, please refer to device_register_request.

3.1.2. HTTP API Signature

fFor HTTP calls, in order to verify the caller, the request content must be signed using the user's private key, and the Base64-encoded signature string must be sent in the request header as "x-SmartWizArt-Signature".

Data to Be Signed	HTTP Request Body
Signature Algorithm	PKCS1v15(SHA256)

3.2. Common Specifications for HTTP APIs

3.2.1. URL

http://<art frame hostname>/api/control/request

Art Frame hostname can be obtained using the BLE "get_status" command.

3.2.2. Request Headers

The request headers must include the following fields:

Content-Type	application/json
x-SmartWizArt-Signature	The Base64-encoded string of the signature generated from the request body.

3.2.3. Request Body

The request body must be in JSON format.

In addition to the fields required by each “Command”, the following fields must always be included:

Field	Data Type	Description
request_id	string	String used to identify the request Any arbitrary string Example: "0001"
request_utc	string	Timestamp for API requests (UTC) A string in ISO 8601 format. Example: 2026-01-01T12:34:56Z
command	string	Command the user wants to execute

3.2.1. Response Body

The response body must be in JSON format.

In addition to the fields required by each “Command”, the following fields must always be included:

Field	Data Type	Description
request_id	string	The accepted request_id
response_utc	string	Timestamp of the response (UTC) A string in ISO 8601 format. Example: 2026-01-01T12:34:56Z
result	boolean	Result of the API operation: true: success false: failure
msg	string	Error message returned when the API call fails.

3.3. Command List

No	Command	Description
1	device_register_request	Registration request to control Art Frame. Exchanges key information for security purposes.
2	device_unregister_request	Request to deregister a registered Art Frame. Deletes the previously exchanged key information.
3	device_config_request	Sets configuration items of Art Frame and retrieves their values.
4	get_device_status_request	Retrieves the status of the Art Frame.
5	image_upload_request	Uploads an image to Art Frame. The data must be converted for use with Art Frame. This command only stores the image data on Art Frame. To display the image, the “display_request” command must be executed.
6	display_request	Requests the display of the uploaded image on Art Frame.

3.4. device_register_request

Registers the user to enable control of the Art Frame.
Exchanges the user's public key with the Art Frame's public key.
Since the Art Frame's public key is used for image encryption, please save it upon receiving the response.

3.4.1. request_fields

Field	Data Type	Required	Description
public_key	string	Yes	User's public key encoded in Base64.

3.4.2. response_fields

Field	Data Type	Description
public_key	string	Art Frame public key encoded in Base64. The user uses this public key to encrypt images.

3.5. device_unregister_request

Deregisters a registered Art Frame.
User data on the Art Frame will be deleted.
The Matter commissioning state will be retained.

3.5.1. request_fields

None

3.5.1. response _fields

None

3.6. device_config_request

Sets configuration items of the Art Frame and retrieves their values.

Specify the items to configure in the request. If only retrieving values, specifying fields in the request is not required.

The response returns the current values of the settings.

3.6.1. request_fields

Field	Data Type	Required	Description
device_name	string	No	Art Frame device name
enable_matter_onoff_on_event	boolean	No	Whether to toggle the display in response to the On state of a Matter Light device.
enable_matter_onoff_on_action	string	No	Display content when the On event occurs. “display-image”, "display-message"
enable_matter_onoff_off_event	boolean	No	Whether to toggle the display in response to the Off state of a Matter Light device.
enable_matter_onoff_off_action	string	No	Display content when the Off event occurs. “display-image”, "display-message"
utc_offset_min	Number	No	UTC offset (in minutes) of the time on Art Frame.

3.6.2. response_fields

Field	Data Type	Description
device_name	string	Art Frame device name
enable_matter_onoff_operation_mode	string	“controlled”(fixed)
enable_matter_onoff_on_event	boolean	Whether to toggle the display in response to the On state of a Matter Light device.
enable_matter_onoff_on_action	string	Display content when the On event occurs. “display-image”, "display-message"
enable_matter_onoff_off_event	boolean	Whether to toggle the display in response to the Off state of a Matter Light device.
enable_matter_onoff_on_action	string	Display content when the Off event occurs. “display-image”, "display-message"
utc_offset_min	integer	UTC offset (in minutes) of the time on Art Frame.

3.7. get_device_status_request

Retrieves the status of Art Frame.

3.7.1. request_fields

None

3.7.2. response_fields

Field	Data Type	Description
model	string	Product model name
device_id	string	Art Frame device ID
device_name	string	Art Frame device name
if_status	object	Network status
ble	boolean	BLE connection status. true:Connected false:Not Connected
wlan	boolean	Connection status of WiFi. true:Connected false:Not Connected
otbr	boolean	whether the Open Thread Border Router is running. true: Running false: Not running
firmware	string	firmware version
battery_status	object	Battery Status
charge	boolean	Charging status. true: Charging in progress false: Not Charging
voltage	integer	battery voltage[mv]
level	integer	battery level
owner_hash	string	Character string used to check the user application that has been registered with the device.
accepted_request_id	string	ID when the request was received from user application.
accepted_request_utc	string	UTC Timestamp when the request was received from user application.
displayed_user_name	string	Name of the user who performed the screen display.
displayed_user_comment	string	Comment set by the user who displayed the screen.
displayed_image_id	integer	Image ID shown in the image file displayed on the screen.
displayed_image_caption	string	Caption data shown in the image file displayed on the screen.
displayed_orientation	integer	device in which the displayed image is supposed to be placed. Values are only as follows: 0 : Horizontal 90: Vertical orientation
utc_offset_min	integer	UTC offset (in minutes) of the time on Art Frame.
updated_info_utc	string	UTC Timestamp when the device status was updated (generated)

3.8. image_upload_request

Uploads an image to be used on the Art Frame.

3.8.1. request_fields

Field	Data Type	Required	Description
display_event_type	string	Yes	display type of the uploaded file. "manual" : User-specified display image "matter_onoff_on": On display image when Matter On /Off function is enabled. "matter_onoff_off":Off display image when Matter On/Off function is enabled. "matter_onoff_msg_on":On display message image when Matter On /Off function is enabled. "matter_onoff_msg_off":Off display message image when Matter On/Off function is enabled.
contents	string	Yes	Base64-encoded image data for the Art Frame.

3.8.2. response_fields

Field	Data Type	Description
file	string	The file name of the uploaded image on Art Frame. Used when requesting display of the image with a “display_request” command.

3.9. display_request

Requests the display of the uploaded image on Art Frame.

3.9.1. request_fields

Field	Data Type	Required	Description
user_name	string	Yes	User who requests the screen display. Uses the username set by the user for the user application. Maximum length is 32 bytes.
user_comment	string	Yes	Comment specified when requesting the screen display. (This is not displayed on the screen.) Maximum character count is 64 bytes or less.

3.9.2. response_fields

None

4. Usage

When using the Art Frame commands, the following operations are required:

- Initialization
- Detecting the Art Frame and retrieving its information
- Connecting to a Wi-Fi network
- Registering the device
- Converting the image to be displayed
- Uploading and rendering the image

This section explains how to use the API as part of the overall workflow, using a Python sample script that demonstrates how to call the API.

When using the API, be sure to supply power to the Art Frame via USB.

If USB power is not provided, the device may enter sleep mode, which could prevent proper communication.

In addition, when issuing commands from the developer tools, initialization (unregistering the Art Frame) is required.

Once initialization (device unregistration) is performed, the Art Frame can no longer be controlled from the smartphone application. Any sharing with friends within the app will also be cancelled.

If you wish to control the Art Frame again from the smartphone application, you must initialize the device and then register the Art Frame again through the smartphone app.

Please note this carefully.

4.1. About dependent libraries and packages

The sample scripts use **ImageMagick**.

Additionally, **pyblufi** (https://github.com/someburner/pyBlufi) is used for Blufi communication.

Please download and install these as needed.

pyBlufi install

```
pip install git+https://github.com/someburner/pyBlufi.git
```

Example: Installing ImageMagick

```
sudo apt update
sudo apt install imagemagick
```

4.2. Initialization

When operating the Art Frame using the developer tools, you must first perform initialization (unregister the Art Frame).

To initialize the device, either select the Art Frame in the smartphone application and execute "Remove this Art Frame", or press and hold the switch on the device for more than 10 seconds (until the LED blinks rapidly).

Once initialization is complete, a 10-digit PID will be displayed on the Art Frame.

4.3. Detecting the Art Device and Retrieving Information

To find nearby Art Frames that can communicate with the tool, execute the scan_art_device command.

If an Art Frame is detected, a list of the detected device's `local_name` values will be output in JSON format.

If multiple devices are found, identify the corresponding Art Frame by checking the 10-digit PID portion of the `local_name`.

If no Art Frame is detected, please check the power status of the device and confirm that the Art Frame is located nearby.

The displayed `local_name` will be used to specify the target Art Frame in subsequent commands, so be sure to make a note of it.

Example

```
python3 ./scan_art_device.py
{"result": true, "devices": [{"local_name": "SMARTWizArt-1234567001"}]}
```

Next, retrieve the Art Frame information.

Execute the get_status command.

The argument should be the local_name obtained using scan_art_device.

Example

```
python3 ./get_status.py SMARTWizArt-1234567001
{"result":true,"device_id":"123456a12345abcde123456789a1234","version":"V1.0.4","model":"
SA073-ES3-B1","h
ostname":"smartwiz-art-123456a12345abcde123456789a1234.local","api_url":"http://smart
wiz-art-123456a123
45abcde123456789a1234.local/api/","wifi_connected":true,"wifi_ssid":"AT-MCM24","is_matt
er_available":false,
"ipv4_addr":"192.168.1.11","ipv6_addr":"FE80::92E5:B1FF:FEAC:CA4","owner_hash":null}
```

If the Art Frame information retrieval is successful, various device details will be returned in JSON format.

Among them, the device_id (e.g., 123456a12345abcde123456789a1234) will be required for executing subsequent commands, so be sure to make a note of it.

4.4. Connecting the Art Device to Wi-Fi

Connect the Art Frame to a Wi-Fi network.

A list of available SSIDs that the Art Frame can connect to can be obtained using the get_ssid_list command. The argument for get_ssid_list is the Art Frame's local_name.

Example

```
python3 ./get_ssid_list.py SMARTWizArt-1234567001
{"result":true,"ssid_list":["MY-HOME-WIFI","Buffalo-G-1234","mobilepoint2",0001docomo","
0000docomo","chimney","ntcm1-38da5c-g"]}
```

The Art Frame must be connected to a network that can communicate with the device running the developer tools.

Once you have confirmed the available SSIDs, specify the SSID and password, and execute the connect_wifi command.

The command arguments are as follows:

・The Art Frames local_name
・The SSID of the network to connect to
・The password for the specified SSID

Example

```
python3 ./connect_wifi.py SMARTWizArt-1234567001 MY -WIFI my_wifi_pass1234
{"result": true, "message": ""}
```

If the Art Frame successfully connects to the Wi-Fi network, the LED on the device will change from white to green.If the connection is not established, the LED will remain white.

4.5. Art Frame Registration

To operate the Art Frame, you must register the device using the developer tools.
Execute the device_register command.
The argument should be the device_id obtained from the get_status command.

Example

```
python3 ./device_register.py 123456a12345abcde123456789a1234
```

While the device is being registered, the LED on the Art Frame will blink.
After the registration is complete, an image displaying the message “Congratulations!” will appear on the device.

Note:
Within the device_register.py script, app_private.der and app_public.der are generated as the user’s private and public keys.
In addition, the art frame’s public key exchanged via the API is saved as epd_public_key.der.
These key files are referenced by other sample scripts as well, assuming they are located in the same directory.

4.6. Converting Images for Display

Since the E Ink Spectra 6 display used in SMARTWIZ+ art is a six-color electronic paper panel, ordinary images cannot be displayed directly.

In order to display images using only six colors, the image data must be processed with dithering and converted into the Spectra 6 data format.

Execute the convert_image command to perform the image conversion.

The command arguments are:

・The file path of the image to be converted (JPG or PNG format)

・The file path for the converted output image

Example

```
python3 ./convert_image.py my_image.png my_image.s6
```

If the conversion completes successfully, the converted file will be saved with the specified filename (e.g., my_image.s6).

The dithered image will also be output with a filename such as my_image.png.dither.png.

You can use this file to preview how the image will appear when displayed on the Art Frame.

The image conversion process uses the ImageMagick package.

Please make sure to install this package in advance.

4.7. Uploading and Displaying an Image

Once the image to be displayed is ready, you can upload and render it on the Art Frame by executing the display_local_image command.

・The command arguments are:
・device_id
・The file path of the converted image data to upload

Example

```
python3 ./display_local_image.py b55bb33a6f832129de818390f1645296 my_image.s6
{'request_id': 'smartwizart-cli-app-0',
'response_utc': '2025-12-24T09:27:46Z',
'response': 'image_upload_request',
'result': True, 'file': 'image_0'}
{'request_id': 'smartwizart-cli-app-1',
'response_utc': '2025-12-24T09:27:48Z',
'response': 'display_request', 'result': True
}
```

Once the upload is completed successfully, the Art Frame will begin rendering the image.

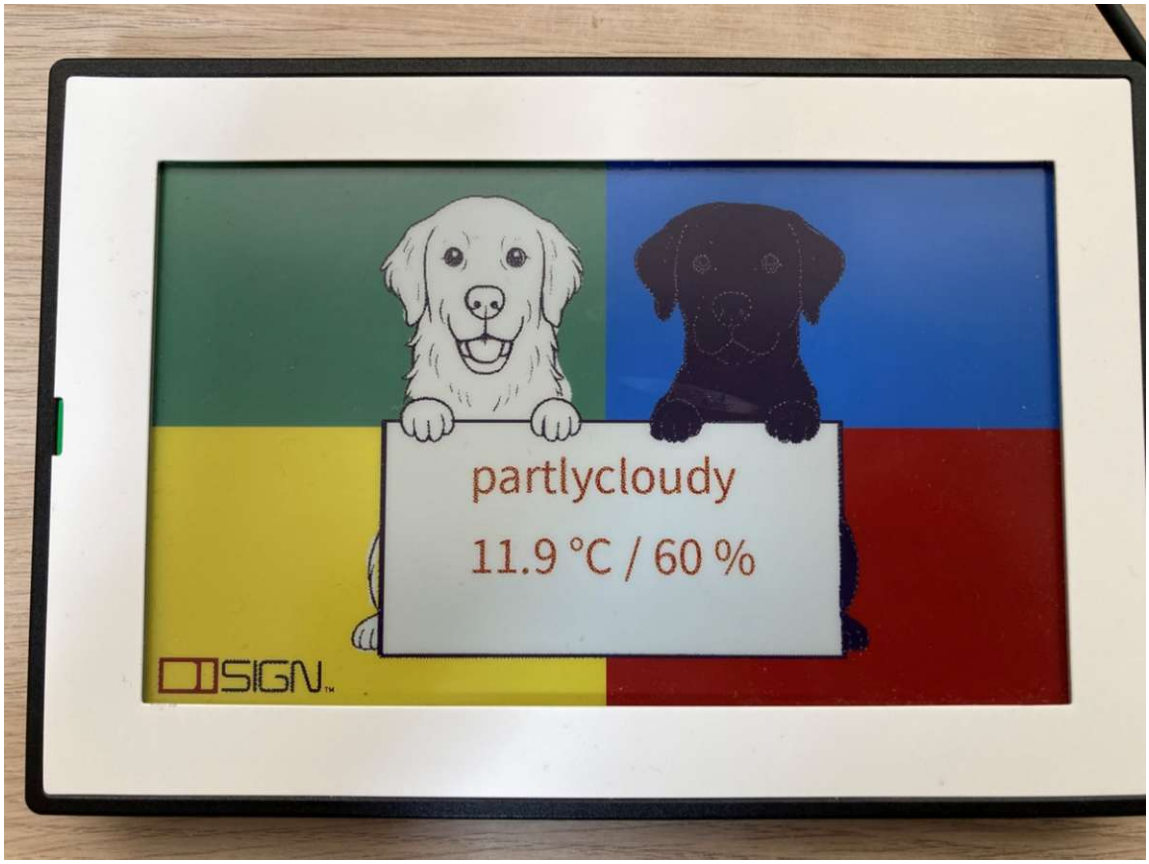
4.8. Integration with Home Assistant

This is an example script that retrieves weather information from Home Assistant and displays the weather, temperature, and humidity on the art frame.

Please export the Home Assistant API access token as HA_ACCESS_TOKEN.
The script argument is device_id.

```
export HA_ACCESS_TOKEN=<your home assistant token>
python3 display_weather_info.py 123456a12345abcde123456789a1234
```

It displays an image like this.



4.9. Unregistering the Art Frame

If the Art Frame has been registered using the developer tools (device_register), it can no longer be controlled from the smartphone application.

To use the Art Frame again from the smartphone app, execute the device_unregister command.

The argument for this command is the device_id.

Example

```
python3 ./device_unregister 123456a12345abcde123456789a1234
```

While the command is being executed, the LED on the Art Frame will blink.

If the unregistration process is successful, a 10-digit PID will be displayed, and art frame will return to the initialized state.

(The Wi-Fi network connection will remain active.)

If this command fails, you can initialize the device by pressing and holding the switch on the Art Frame for more than 10 seconds.

5. Precautions for Use

No.	Item	Description
1	Image Upload Frequency	Do not repeatedly upload and display images at short intervals for extended periods, as this may shorten the lifespan of the Art Frame.